



NTSB Accident Investigation Report Analysis

Kanu Shetkar & Sankaran Mahadevan

Dr. Maha meeting 1/21/26

Agenda

- NTSB background & data
- Prior paper (Maha & Zhang 2021)
- Bayesian Networks: simple example
- Previous approach and Limitations
- Current approach
- Model structure overview
 -
- Demo
- Evaluation, Limitations, Future Work



NTSB Overview

National Transportation Safety Board:

- Independent U.S. agency (est. 1967)
- Investigates transportation accidents



Prior Paper



- Bayesian Network on NTSB coded data
- Backward inference: infer likely causes from outcomes
- Forward inference: conditional probabilities of downstream events

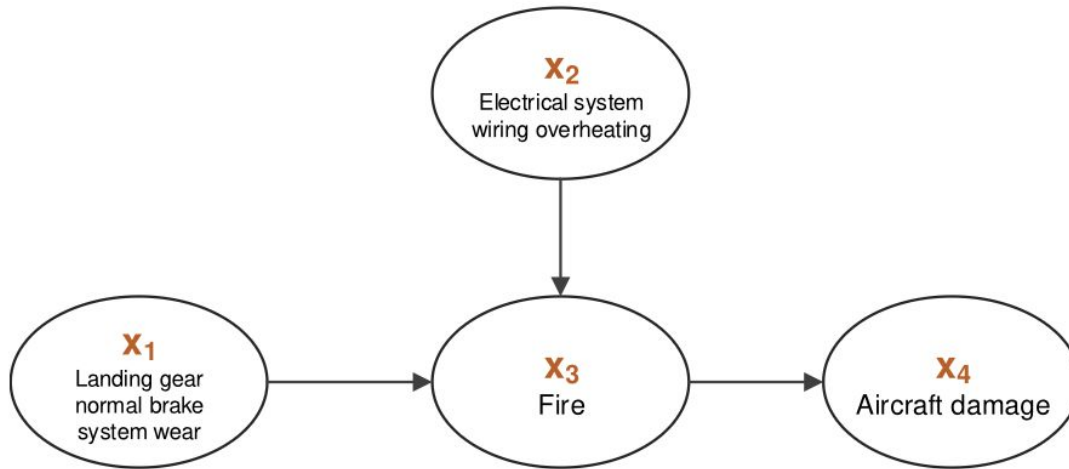
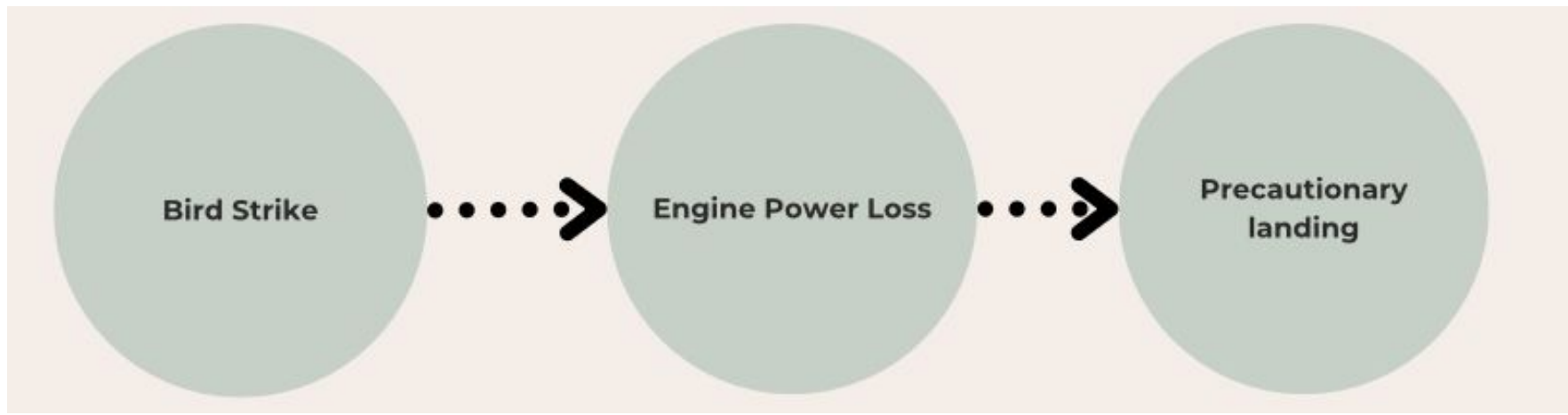


Fig. 2. A simple Bayesian network illustrating the occurrence of an aviation accident.



What is a Bayesian Network?

- Nodes = events/conditions
- Arrows = dependencies (cause → effect)
- Inference = forward or backward
- $P(\text{Engine Power Loss} \mid \text{Bird Strike}) = 0.40$



Previous Approach

- Pure gpt o4
- Goal was to see how much the LLM could do on its own given prompting



Limitations

- LLM inference is not the same as explicit Bayesian probabilities
- Risk of hallucinations or overconfident outputs
- Limited validation so far (demo-level testing)
- Sensitive to input phrasing and prompts
- Computational challenges at scale





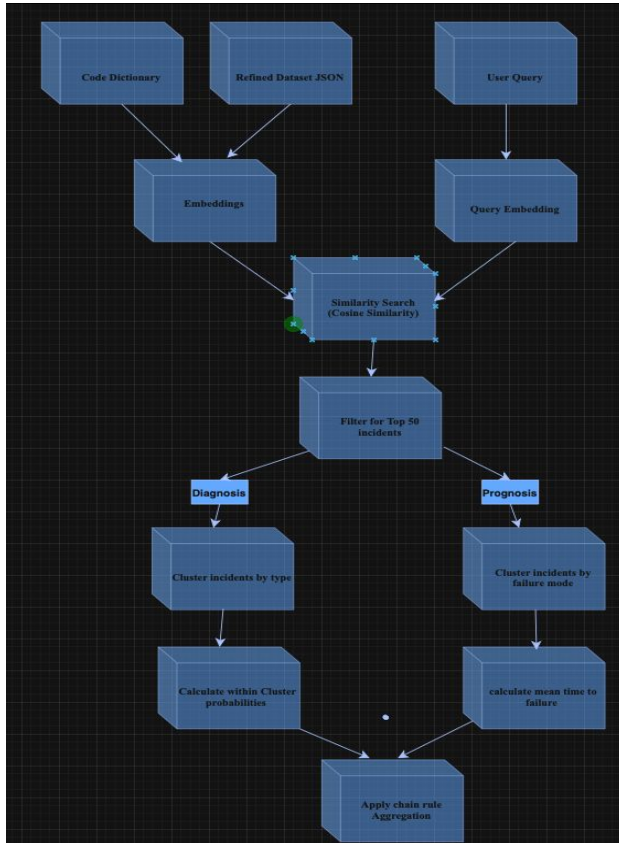
Expanding Prior Work

- Same dataset & timeframe (1982–2016, FAR 121, accidents)
- Prior work: coded data + BN inference
- My prior work: pure LLM approach
- My extension: coded + narratives (structured + unstructured)
- LLM-based reasoning → backward & forward inference

Model Structure Overview



- **Diagnosis Mode:**
 - Analyzes completed incidents to understand what caused accidents
 - identify prevention strategies for precursors
- **Prognosis Mode:**
 - Forecasts how current symptoms might escalate into future failures
 - suggest preventive actions for downstream events



Refined Dataset JSON



- Data is scattered in 14 different files
 - Aircraft, sequence of events, engines, findings, injury, narratives, and occurrences file
 - In .txt and .xlsx files
- Last semester we were looking at each one of these files
 - Result: this was taking a lot of time
- Solution: We merged all files using common EvID column name into a refined Dataset JSON file
 - This will allow for faster retrieval in results when running the streamlit app
 - Next slide shows what the dataset looks like
- Total: ~2,243 incident reports

```

"20001212X23783": {
  "ev_date": "1990-07-29T00:00:00",
  "ev_time": 1757.0,
  "ev_city": "NEW YORK",
  "ev_state": "NY",
  "ev_country": "USA",
  "light_cond": "DAYL",
  "sky_cond_nonceil": "SCAT",
  "vis_sm": 20.0,
  "wx_temp": -18.0,
  "wind_dir_deg": 160.0,
  "wind_vel_kts": 8.0,
  "gust_kts": 0.0,
  "wx_cond_basic": "VMC",
  "ev_highest_injury": "NONE",
  "narr_accp": null,
  "narr_accf": "THE FLIGHT CREW WAS UNABLE TO",
  "narr_cause": "THE FLIGHT CREW'S FAILURE TO",
  "damage": "MINR",
  "acft_fire": "NONE",
  "acft_expl": "NONE",
  "acft_make": "DE HAVILLAND",
  "acft_model": "DHC-7-102",
  "total_seats": 54.0,
  "num_eng": 4.0,
  "type_last_insp": "UNK",
  "afm_hrs": null,
  "sequence_of_events": [],
  "findings": [],
  "injuries": [
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "ABRD",
      "injury_level": "NONE",
      "inj_person_count": 45.0
    },
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "ABRD",
      "injury_level": "TOTL",
      "inj_person_count": 45.0
    },
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "CABN",
      "injury_level": "NONE",

```

```

"20001212X23783": {
  "injuries": [
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "CPLT",
      "injury_level": "TOTL",
      "inj_person_count": 1.0
    },
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "PAX",
      "injury_level": "NONE",
      "inj_person_count": 42.0
    },
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "PAX",
      "injury_level": "TOTL",
      "inj_person_count": 42.0
    },
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "PLT",
      "injury_level": "NONE",
      "inj_person_count": 1.0
    },
    {
      "ev_id": "20001212X23783",
      "inj_person_category": "PLT",
      "injury_level": "TOTL",
      "inj_person_count": 1.0
    }
  ],
  "engines": [
    {
      "ev_id": "20001212X23783",
      "eng_no": 1,
      "eng_type": "TP",
      "eng_mfgr": "P&W",
      "eng_model": "PT6-50",
      "power_units": 1120.0,
      "propeller_type": null
    }
  ],
  "cluster_label": "Hydraulic Failure"

```



- Example of what it looks like
 - (just for Dr. Maha eyes)

Embeddings



- We pass the Refined dataset and the User query through OpenAI's text-embedding-3-small model
- Why: it converts text descriptions into high dimensional numerical vectors (1,536 dimensions per narrative)
- This allows the system to understand semantic meaning
- Each incident becomes a point in vector space; similar incidents cluster together
- Ex: "Engine failure during cruise" < – > "Loss of power in flight" = High similarity
- "Engine failure during cruise" < – > "Bird strike on runway" = Low similarity

Similarity Search(Cosine similarity) & filter for top 50 incidents



- Goal: Find the most relevant historical incidents for any query
- Method: cosine similarity → measures how close 2 descriptions are
- Thing of it like this: comparing the directions of 2 arrows in space
 - Same direction = 1 (identical meaning)
 - Opposite direction = 0 (different meaning)
 - Angled = 0.5-0.9 = (somewhat similar)
- Ex result:
 - Query: engine lost power during flight
 - 0.87: engine failure in cruise → highly relevant
 - 0.82: total power loss → very relevant
 - 0.25: bird strike on ground → not relevant
- Process: compare query against all 4,360 incidents, take top 50
- Formula: $\text{similarity}(q,i) = \text{dot_product}(q,i) / \text{magnitude}(q) \times \text{magnitude}(i)$
 - Where
 - q = query embedding vector
 - i = incident embedding vector
 - Dot product = multiply each pair of elements, then sum
 - Magnitude = length of vector = $\sqrt{\text{sum of squared elements}}$
 - Output: 0(unrelated) → 1 (identical meaning)

Clustering



- Problem:
 - Top 50 incidents are a mix of different failure scenarios
 - Need to organize by incident type before calculating probabilities
- Setup phase: (done once)
 - K means clustering on all 2,243 incidents into 50 cluster labels
 - LLM names each cluster (ex: engine failure, fuel system)
 - Cluster labels are saved to each incident in the refined dataset
- Query phase (fast lookup)
 - Retrieving top 50 similar incidents
 - Look up their pre computed cluster labels
 - Group the 50 by the cluster type
- Example query result:
 - 12 incidents → engine failure cluster
 - 8 incidents → fuel system cluster
 - 5 incidents → landing gear cluster
 -



Finding cause Frequencies in each Cluster

- Step 2: Calculate Cause probabilities within each cluster
- What we are calculating:
 - $P(\text{Cause} \mid \text{Cluster})$ = given this type of incident, how likely is this cause?
- Where causes come from:
 - NTSB structured findings (ex: cause factor: improper maintenance)
 - Narrative text extraction (phrases like due to fuel contamination)
 - Each incident can have multiple causes listed
- Process:
 - 1. Take one cluster (ex: engine failure with 12 incidents)
 - 2. Extract all causes from those 12 incidents
 - 3. Count frequency of each unique cause
 - 4. Calculate probability = count / total



Example: Engine failure cluster

- Concrete example: Engine fair cluster
 - 12 incidents retrieved
 - Total cause extracted: 20 (some incidents have 1 cause, some have 3)
- Cause frequencies:
 - Improper maintenance → appears 7 times
 - Fuel contamination → appears 5 times
 - Pilot error → appears 2 times
 - Manufacturing defect → appears 3 times
- Calculations:
 - $P(\text{improper maintenance} \mid \text{engine failure}) = 7/20 = 0.35$ (35%)
 - $P(\text{fuel contamination} \mid \text{engine failure}) = 5/20 = 0.25$ (25%)
 - $P(\text{pilot error} \mid \text{engine failure}) = 3/20 = 0.15$ (15%)



Apply chain rule

- Goal: calculate final $P(\text{Cause} \mid \text{Query})$ across all clusters
- Formula: $P(\text{cause} \mid \text{query}) = \sum P(\text{Cause} \mid \text{Cluster}) \times P(\text{Cluster} \mid \text{Query})$ for all clusters
- 2 key pieces
 - $P(\text{cause} \mid \text{cluster}) \rightarrow$ already calculated (35%, 25%, etc)
 - $P(\text{cluster} \mid \text{query}) \rightarrow$ how relevant is each cluster to this query?
- How we calculate $P(\text{Cluster} \mid \text{Query})$:
 - Weight = Avg similarity x Number of incidents in cluster
 - $P(\text{Cluster} \mid \text{Query}) = \text{Cluster weight} / \text{total weight across all clusters}$
- Ex:
 - engine failure cluster: avg similarity 0.85, 12 incidents
 - Weight = $0.85 \times 12 = 10.2$
 - If total weight = 30, then $P(\text{engine failure} \mid \text{Query}) = 10.2 / 30 = 0.34$ (34%)



Chain Rule - Worked Example

Complete example: calculating $P(\text{improper maintenance} \mid \text{Query})$

Setup: top 50 incidents grouped into 3 clusters

Engine failure: 12 incidents, avg similarity = 0.85

Fuel system: 8 incidents, avg similarity = 0.75

Landing gear: 5 incidents, avg similarity = 0.60

Step 1: Calculate $P(\text{Cluster} \mid \text{Query})$ for each cluster

Cluster Weights:

- Engine Failure: $0.85 \times 12 = 10.2$
- Fuel System: $0.75 \times 8 = 6.0$
- Landing Gear: $0.60 \times 5 = 3.0$
- Total Weight = $10.2 + 6.0 + 3.0 = 19.2$

continued



Step 2: Get $P(\text{"Improper Maintenance"} | \text{Cluster})$ from earlier calculation

- $P(\text{Improper Maintenance} | \text{Engine Failure}) = 0.35$ (35%)
- $P(\text{Improper Maintenance} | \text{Fuel System}) = 0.20$ (20%)
- $P(\text{Improper Maintenance} | \text{Landing Gear}) = 0.10$ (10%)

Step 3: Apply Chain Rule

$$\begin{aligned} P(\text{"Improper Maintenance"} | \text{Query}) &= \\ & (0.35 \times 0.53) + (0.20 \times 0.31) + (0.10 \times 0.16) \\ &= 0.186 + 0.062 + 0.016 \\ &= 0.264 = 26.4\% \end{aligned}$$

Final Answer: "Improper Maintenance" has 26.4% probability as root cause

Live Demo





Thank you!

Appendix





Clustering

- Problem: raw similarity gives us a mix of incidents. We need to organize them
- Solution: used LLM GPT-4o-mini to sort top 50 incidents into distinct groups on the fly

Diagnosis:

- Group by Incident type
- Goal: create scenarios to check for root causes

Prognosis:

- Group incidents by failure mode
- Goal: create scenarios to check for component lifespan



Describe Incident

- Input: free-text narrative describing the incident
- Narrative becomes the structured query for analysis

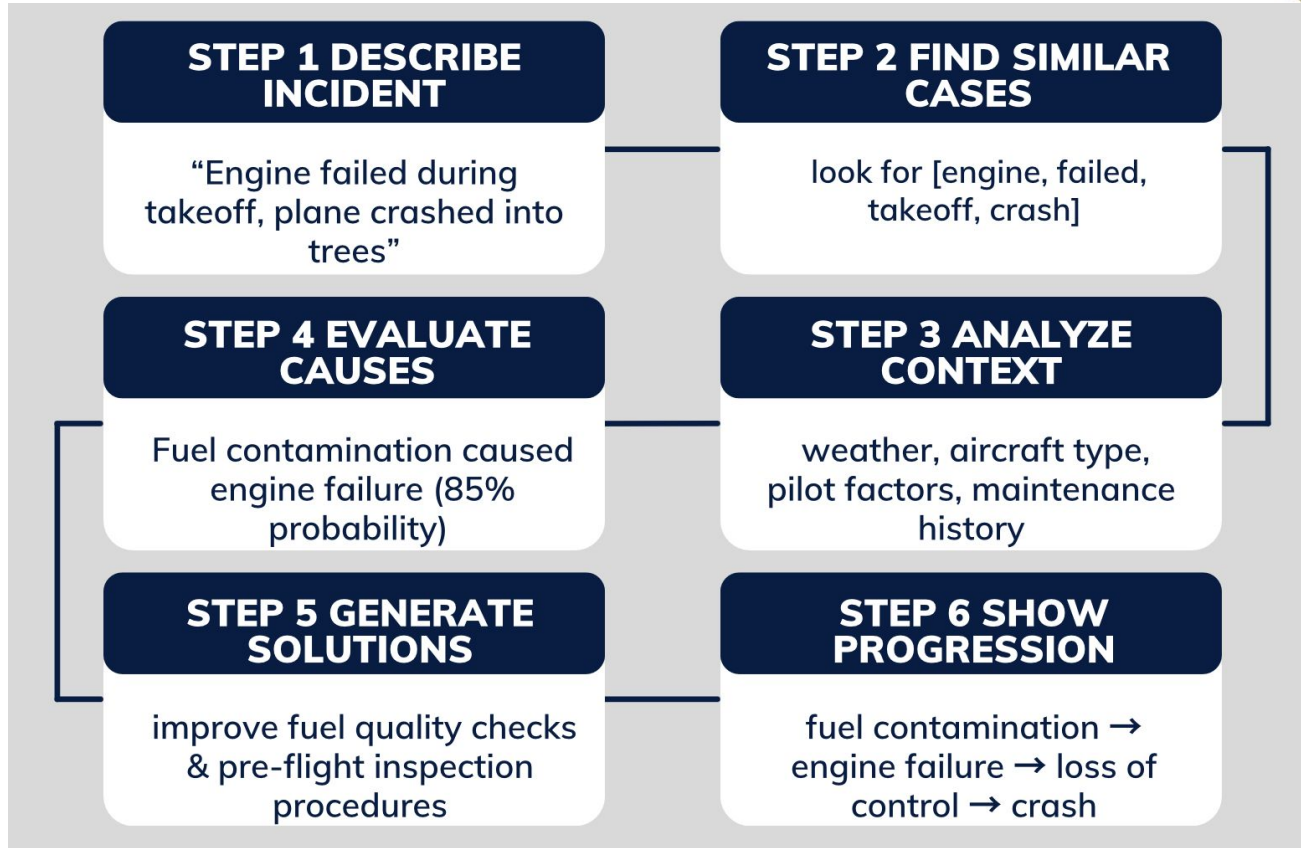
Incident Description

Describe the aviation incident:

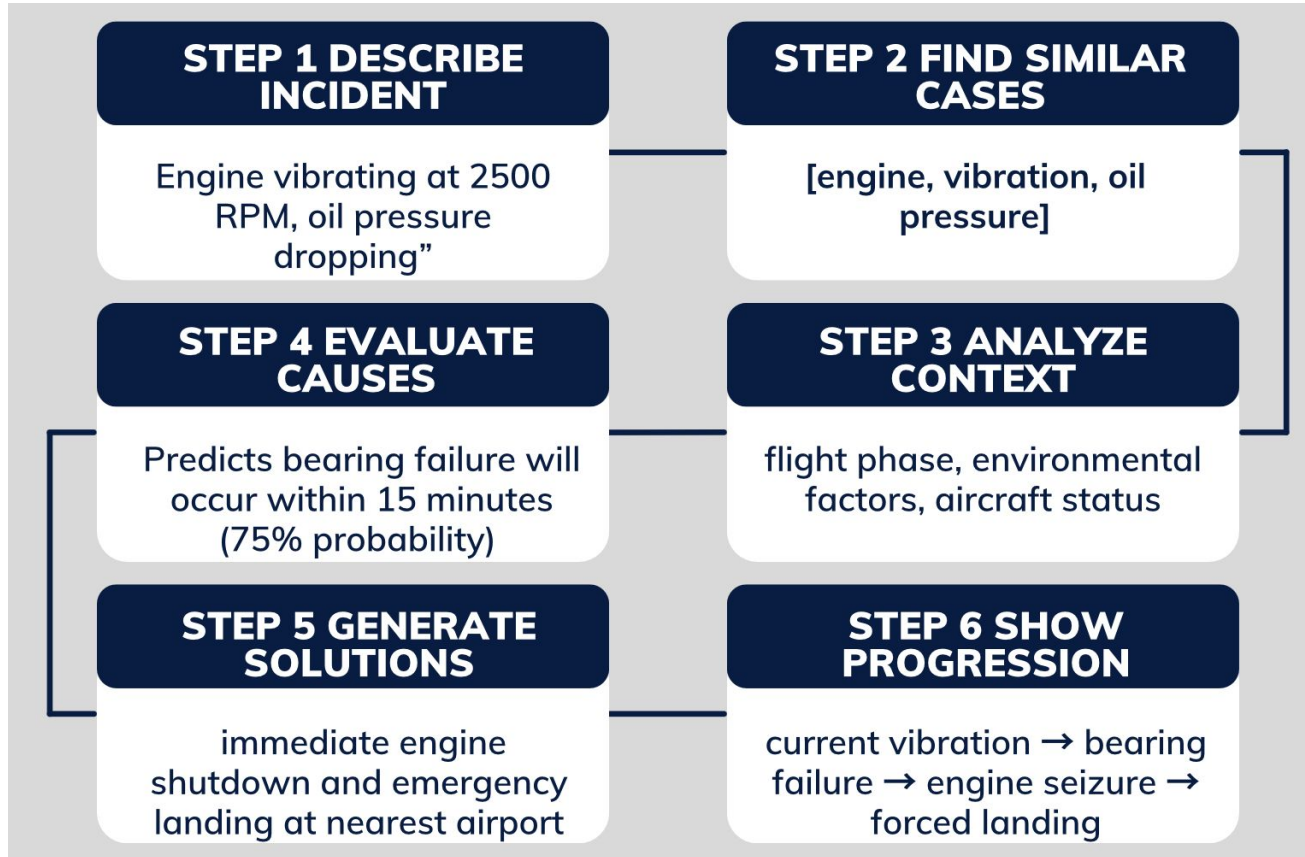
Air India Flight 171, a Boeing 787 Dreamliner, crashed shortly after takeoff from Ahmedabad, India on June 12, 2025. The aircraft was heading to London Gatwick and carried 242 people. Only one person survived. Why did this crash happen?

Analyze Incident

Diagnosis mode



Prognosis mode



Data



- ~25,000 accident records
- Detailed narratives: pilot/crew descriptions of what happened
- Event file: step by step chains showing how incidents escalated to final outcomes
- Code dictionary: translates numeric codes into aviation meaning
 - (e.g. 1452 → pilot error)
- Aircraft & engine file: performance history, maintenance records, and operational details
- Findings file: early warning signs and precursor conditions that led to incidents
- Occurrences file: timing and progression details for each accident phase

Future Tasks



- Test Zhang & Dr. Maha's examples against my LLM system
- Experiment with multiple LLMs (GPT & Claude) for improving reasoning
- Update prompting for both the Diagnosis & Prognosis Modes
- Add data saving and export features
- Validate accuracy with more test cases
- Explore hybrid models: combine BN-style probabilities with LLM narrative reasoning



